

Original articles

An algorithm for single- and multiple-runway aircraft landing problem

Amir Salehipour

School of Mathematical and Physical Sciences, University of Technology Sydney, Australia

Received 24 May 2018; received in revised form 16 August 2019; accepted 12 October 2019

Available online 22 October 2019

Abstract

The aircraft landing problem (ALP) is the problem of allocating an airport's runways to arriving aircraft as well as scheduling the landing time of aircraft, with the objective of minimizing the total deviation from the target landing times. We propose a simple heuristic to solve ALP. The distinguishing factor of the proposed algorithm includes decomposing the problem into a chain of smaller and easier-to-solve problems. We show that utilizing this strategy is very effective in solving the problem in a short time. Our work is motivated by the dynamic nature of the problem, i.e., due to the continuous changes in the number of arriving flights and a short window for determining the landing schedule, the air traffic controller needs to solve the problem on a regular basis and update the landing schedule, and fast and effective algorithms are therefore paramount. By solving two sets of 124 benchmark instances we demonstrate that we fulfill this aim and that the proposed algorithm obtains satisfactory solutions in a short amount of time.

© 2019 International Association for Mathematics and Computers in Simulation (IMACS). Published by Elsevier B.V. All rights reserved.

Keywords: Earliness–tardiness; Sequence relaxation; Runway scheduling; Aircraft sequencing

1. Introduction

The aircraft landing problem (ALP) is an important class of scheduling problems and has broad applications in the area of transportation, providing significant benefits to systems experiencing long delays. ALP is the problem of allocating an airport's runways to arriving aircraft as well as scheduling the landing time of aircraft. The amount of deviation from the target landing times between the earliest and latest landing times is the basis for determining the penalty. The objective of ALP is to minimize the penalized deviations from the target landing times. Hence, the objective function includes two components: penalizing early and late landings. In practice, the penalty per unit of early or late landing is a function of aircraft type, passenger capacity and fuel economy, among others [8].

A safe landing must ensure the minimum separation between every ordered pair of aircraft. The two most applied separation requirements are “radar separation”, which is a lengthwise spacing of five nautical miles (some busy airports may consider less than this) and a vertical spacing of 1000 ft, and “wake turbulence separation” (also known as wake vortex), which is a time spacing and depends on the type of aircraft. The international civil aviation organization (ICAO) considers four categories of light (L), medium (M), heavy (H) and super (S) for wake

E-mail address: amir.salehipour@uts.edu.au.

<https://doi.org/10.1016/j.matcom.2019.10.006>

0378-4754/© 2019 International Association for Mathematics and Computers in Simulation (IMACS). Published by Elsevier B.V. All rights reserved.

turbulence (category S only includes Airbus A380). Other factors such as aircraft route and weather condition may impact the separation between aircraft.

Other operational constraints for ALP may include earliest and latest time windows for landings, runway suitability for specific landings and constrained position shifting. The latter is used to model the “fairness policy” among airlines [3] and implies that each aircraft in the sequence may only deviate by a certain number of positions from its position in the first-come-first-serve (FCFS) order. FCFS is a routine administered by the air traffic controllers, in which the aircraft are sequenced for landing according to their arrival time to the airport. However, significant improvements are possible with the use of optimization techniques. This study contributes to this aim by presenting optimization algorithms for ALP.

Available methods for solving ALP can be grouped into exact and heuristics. Ernst et al. [7] proposed an exact simplex-based algorithm and obtained optimal solution for instances with up to 44 aircraft. Bianco et al. [6] modeled the problem on one runway as a single machine scheduling problem with ready times, sequence-dependent setup times and objective function of minimizing the completion time. A mixed-integer program was developed by Beasley et al. [4]; the authors managed to solve instances with up to 50 aircraft to optimality. Later, several algorithms including branch-and-price and column generation were proposed for ALP [9,10,20]. Among heuristic and meta-heuristic algorithms, the first effort is due to Abela et al. [1], who proposed a genetic algorithm (GA) to solve small instances. Other notable studies include the work of Salehipour et al. [17], Vadlamani and Hosseini [19], Sabar and Kendall [14] and Girish [11]. Those studies investigated advanced meta-heuristics for ALP and reported quality solutions. Recently, Salehipour and Ahmadian [15] and Salehipour et al. [16] developed several algorithms including novel relaxation neighborhoods, and overcame some of the limitations of the previous heuristic algorithms, though they did not consider multiple runways. In general, the available solution methods are very effective when the number of runways increases. Several special cases of ALP have also been studied. For example, Salehipour et al. [18] investigated a case in which the separation time between an ordered pair of aircraft is a function of the runways.

This study contributes to solving ALP by developing simple and efficient algorithms, through changing the traditional views towards modeling and solution methods. Our solution methods, although relying on exact solvers, significantly enhance the capabilities of those solvers in delivering quality solutions for ALP in a short time. Without applying the proposed solution methods, exact solvers are unable to solve the problem in a reasonable amount of time, which is important for real-time and practical applications. Indeed, our work is motivated by the dynamic nature of ALP, i.e., due to the continuous changes in the number of arriving flights and a short window for determining the landing schedule, the air traffic controller will be required to re-solve the problem on a regular basis and update the landing schedule, and fast and effective algorithms are therefore crucial. Available exact methods, though they were used in a heuristic way by setting a time limit, are not able to deliver good quality solutions in a short amount of time [4,9,10,20]. In addition, they can only produce optimal solution for small instances. For example, over tested instances in Section 4 under the single runway setting, the optimal solution is only known for instances with up to 50 aircraft. On the other hand, existing heuristic methods do not guarantee solution’s quality, nor do they have robust performance, because they include randomized elements (see for example [11,13,17]). In addition, because they include a combination of different neighborhood structures as well as hybrid algorithms, their implementation requires advanced algorithmic techniques and parameters tuning.

We believe that the most important advantages of the algorithm proposed in this study include (1) its conceptual simplicity, (2) its ability to deliver high quality solutions for large instances in a short time, (3) its ease of implementation, and (4) its usability for practical and online applications. Our algorithm has only a few parameters, allowing it to be easily tuned and implemented. In addition, we utilize available optimization solvers such as Gurobi because they have greatly improved in recent years, making solver-based algorithms significantly faster and more effective.

The remainder of the paper is organized as follows. Section 2 defines the problem, notations and formulates ALP as a mixed-integer program. Section 3 proposes a construction and a heuristic method for ALP. The section is followed by Section 4, which reports the computational outcomes. Finally, Section 5 concludes the paper.

2. Problem statement

Given a set of aircraft $I = \{1, \dots, n\}$ and runways $R = \{1, \dots, m\}$, the aim of ALP is to obtain a sequence for aircraft landings and the schedule for those landings such that the penalties associated with early and late landings

Table 1

Mathematical notations used in the paper.

Sets	
I	The set of aircraft, $I = \{1, \dots, n\}$, $ I = n$.
R	The set of runways, $R = \{1, \dots, m\}$, $ R = m$.
Parameters	
s_{ij}	Separation time units between two ordered aircraft i and j when landing on the same runway; $s_{ij} > 0$, $i, j \in I$, $i \neq j$.
T_i	Target landing time of aircraft i , $T_i \geq 0$, $i \in I$.
E_i	Earliest landing time of aircraft i , $E_i \geq 0$, $i \in I$.
L_i	Latest landing time of aircraft i , $L_i \geq 0$, $L_i > E_i$, $i \in I$.
c_i^-	Cost of early landing of aircraft i , $c_i^- > 0$, $i \in I$ per unit of earliness.
c_i^+	Cost of late landing of aircraft i , $c_i^+ > 0$, $i \in I$ per unit of lateness.
Variables	
x_i	Scheduled landing time of aircraft i , $x_i \geq 0$, $i \in I$.
α_i	Earliness of landing aircraft i (landing before target landing time), $\alpha_i = \max(0, T_i - x_i)$, $i \in I$.
β_i	Lateness of landing aircraft i (landing after target landing time), $\beta_i = \max(0, x_i - T_i)$, $i \in I$.
y_{ij}	Whether aircraft i lands before aircraft j , $y_{ij} \in \{0, 1\}$, $i, j \in I$, $i \neq j$.
δ_{ij}	Whether aircraft i and j land on the same runway, $\delta_{ij} \in \{0, 1\}$, $i, j \in I$, $i \neq j$.
γ_{ir}	Whether aircraft i is allocated to runway r , $\gamma_{ir} \in \{0, 1\}$, $i \in I$, $r \in R$.

are minimized. Therefore, ALP aims to schedule the landing of all aircraft as close as to their “target landing time”. The target landing time is typically calculated based on the planned arrival route, cruise speed, i.e., the most economical or preferred speed, and the standard descent profile of the aircraft [5]. Fuel, ground handling and transit and connection considerations, and avoiding congestion at airports are among important motivations for scheduling flights arrival around their target landing times.

The early landing of aircraft i is given by $\alpha_i = \max(0, T_i - x_i)$ when aircraft i lands earlier than time T_i , that is, if $x_i \leq T_i$, where x_i is the scheduled (planned) landing time of aircraft i and T_i is the target landing time of aircraft i . The late landing of aircraft i is given by $\beta_i = \max(0, x_i - T_i)$ when aircraft i lands later than time T_i , that is, if $x_i \geq T_i$. Typically, aircraft i is required to land in the time window $[E_i, L_i]$, i.e., $E_i \leq x_i \leq L_i$, where E_i and L_i are the earliest and latest landing times of aircraft i . Note that $\alpha_i > 0$ if the decision variable x_i lies within the range $[E_i, T_i]$, while $\beta_i > 0$ if it lies within the range $[T_i, L_i]$. The reader may realize that capturing two penalties results in a piecewise objective function.

There is a safety separation time $s_{ij} > 0$ between every pair of aircraft $i, j \in I$, $i \neq j$ if they land one after another on the same runway. The separation time can be different for different pairs of aircraft. We assume that the separation time between a pair of aircraft landing on different runways is 0. This assumption has been taken in previous studies as well: for two examples see [11,13]. Table 1 summarizes the mathematical notations used in this paper.

2.1. Mathematical formulation

ALP can be formulated as a mixed-integer program. Problem P shows this, which is due to Beasley et al. [4]. Salehipour et al. [17] provided a simpler version of problem P.

Problem P

$$\min z = \sum_{i=1}^n (\alpha_i c_i^- + \beta_i c_i^+) \quad (1)$$

Subject to

$$E_i \leq x_i \leq L_i, \forall i \in I, \quad (2)$$

$$x_i - T_i = \beta_i - \alpha_i, \forall i \in I, \quad (3)$$

$$x_j - x_i \geq s_{ij} \delta_{ij} - M y_{ji}, \forall i, j \in I, i \neq j, \quad (4)$$

Table 2
Aircraft landing data of the example.

Aircraft	E_i	T_i	L_i
1	129	155	559
2	195	258	744
3	89	98	510
4	96	106	521
5	110	123	555
6	120	135	576
7	124	138	577
8	126	140	573
9	135	150	591
10	160	180	657

$$y_{ij} + y_{ji} = 1, \forall i, j \in I, i \neq j, \quad (5)$$

$$\delta_{ij} \geq \gamma_{ir} + \gamma_{jr} - 1, \forall i, j \in I, i \neq j, \forall r \in R, \quad (6)$$

$$\sum_{r=1}^m \gamma_{ir} = 1, \forall i \in I, \quad (7)$$

$$y_{ij}, \delta_{ij}, \gamma_{ir} \in \{0, 1\}, \forall i, j \in I, i \neq j, \forall r \in R, \quad (8)$$

$$x_i, \alpha_i, \beta_i \geq 0, \forall i \in I. \quad (9)$$

In problem P, the objective function (Eq. (1)) minimizes the total cost of landing deviation from the target landing times, where c_i^- is per unit penalty of early landing of aircraft i and c_i^+ is per unit penalty of late landing of aircraft i . Constraints (2) ensure that every aircraft lands in its time window. Constraints (3) link the decision variables x_i and parameters T_i to decision variables α_i and β_i . Constraints (4) impose that if two aircraft i and j could land on the same runway, at least s_{ij} time units should be elapsed before aircraft j could land on that runway. Given a pair of aircraft, constraints (5) ensure one lands before the other. Constraints (6) link the decision variables δ_{ij} and γ_{ir} . Constraints (7) ensure that every aircraft lands on only one runway. Constraints (8) and (9) ensure that decision variables y_{ij} , δ_{ij} and γ_{ir} only take binary values and decision variables x_i , α_i and β_i only take non-negative values. In problem P, M is a constant taking a large value, which can numerically be calculated from the data. Beasley et al. [4] showed that M can be calculated as $L_i + s_{ij} - E_j$. We replace M by $L_i + s_{ij} - E_j$ when solving ALP instances in Section 4.

2.2. An illustrative example

Now consider the following example, which includes 10 aircraft and two runways, i.e., $n = 10$ and $m = 2$, and represents the instance “Airland1” from OR Library at <http://people.brunel.ac.uk/~mastjjb/jeb/info.html>. Table 2 details the landing data of the aircraft and Table 3 shows the separation times between every pair of aircraft landing on the same runway. Recall that in this study we assume a value of 0 for the separation time between a pair of aircraft landing on two different runways. Also, for the sake of this example, assume that the earliness and lateness costs per time unit of deviation from T_i are 1. An intuitive landing sequence for this example is then $\pi = (3, 4, 5, 6, 7, 8, 9, 1, 10, 2)$, which is obtained by ordering the aircraft landings on non-decreasing order of their target landing times.

At the beginning of scheduling, both runways are available for landings. Therefore, aircraft 3 is the first to land on runway 1 followed by aircraft 4. Hence, $s_{34} = 8$, implying that aircraft 4 cannot land earlier than time 106 on runway 1, which turns out to be its target landing time. Similarly, aircraft 5 and 6 can land on runway 1 at their target landing times. Thus, no deviation from their target landing times has occurred. Aircraft 7, however, cannot

Table 3

The separation times between every pair of aircraft of the example.

i	1	2	3	4	5	6	7	8	9	10
1	–	3	15	15	15	15	15	15	15	15
2	3	–	15	15	15	15	15	15	15	15
3	15	15	–	8	8	8	8	8	8	8
4	15	15	8	–	8	8	8	8	8	8
5	15	15	8	8	–	8	8	8	8	8
6	15	15	8	8	8	–	8	8	8	8
7	15	15	8	8	8	8	–	8	8	8
8	15	15	8	8	8	8	8	–	8	8
9	15	15	8	8	8	8	8	8	–	8
10	15	15	8	8	8	8	8	8	8	–

land earlier than time 143 on runway 1. This would impose 5 time units of delay for this aircraft. We note that if aircraft 7 lands on runway 2 (different runway from that of aircraft 6), it can land at its target landing time, i.e., at time 138 and no delay therefore is experienced for aircraft 7. In this particular example, it would be beneficial to force aircraft 6 and 7 to land on two different runways.

3. The proposed solution method

We propose a heuristic solution method for solving ALP. The proposed heuristic starts from a sequence of aircraft landings and operates by decomposing the original ALP into a chain of “easier-to-solve” problems. We note that if a landing sequence is known, problem P becomes a linear program since a given landing sequence leads to fixing all binary variables y_{ij} and δ_{ij} . Therefore, if the landing sequence is given, we can deliver an optimal schedule in polynomial time. Such an optimal landing schedule can be delivered by utilizing available solvers, e.g., Gurobi [12]. Also, when a landing sequence is manipulated, e.g., through changing position of two aircraft, it is not difficult to obtain the optimal schedule for the new sequence.

We generate an initial landing sequence by applying the construction method of Salehipour et al. [17]. The heuristic uses a simple, yet effective sequence manipulation operator in order to obtain new (improved) landing sequences. The manipulation operator changes a given sequence by letting a chain of consecutive aircraft, i.e., a chunk, be optimized. This may lead to a new order of aircraft landings for the chunk. We utilize standard exact solvers to deliver the schedule for the sequence. Next, we explain constructing an initial sequence and then we detail the heuristic algorithm.

3.1. Generating an initial sequence

In order to generate an initial sequence for ALP we apply the construction method of Salehipour et al. [17]. This algorithm obtains an initial sequence of landings by applying the earliest target landing time (ETLT) first dispatching rule. ETLT sorts the aircraft in non-decreasing order of their target landing times. Therefore, the aircraft with the earliest target landing time lands first, followed by the one with the second earliest target landing time. On multiple runway instances, ETLT assigns a pair of aircraft to two different runways if the succeeding aircraft in the pair cannot land on its target landing time, but later. The algorithm stops when all aircraft are positioned to land, implying a landing sequence is developed. We note that because ETLT ensures that aircraft are sequenced according to their target landing times, which lie between the earliest and latest landing times, feasible sequences are delivered. We refer the interested reader to Salehipour et al. [17] for details.

3.2. The heuristic algorithm

In this section we propose a heuristic algorithm for solving ALP on both single and multiple runways. The algorithm starts from an initial sequence, which is generated by ETLT and iteratively chooses a chunk of aircraft, i.e., a small number of aircraft, relaxes their position in the sequence and optimizes their position through solving

problem P by an exact solver. Indeed, the heuristic lets a small number of aircraft that are in the chunk be repositioned within the chunk, while keeping the current position for the aircraft that are not in the chunk. We let integer parameter k denote the size of the chunk.

The heuristic then optimizes problem P by letting the variables y_{ij} associated with the aircraft in the chunk take binary values, and ensuring that the remaining variables y_{ij} , i.e., those associated with the aircraft that are not in the chunk take either a value of 0 or 1 depending on their position in the sequence. This leads to a smaller number of binary variables y_{ij} in problem P, resulting in an easier problem to solve. We are particularly interested in keeping the size of the chunk small, i.e., only a few aircraft are subject to optimization. This is because the exact solvers may then deliver near-optimal (and possibly the optimal depending on the size of the chunk) sequence and schedule of landings for the aircraft in the chunk. By starting from the beginning of a given sequence and iterating to the end of the sequence, the heuristic solves a set of $\lceil \frac{n}{k} \rceil$ problems P. The algorithm keeps generating and optimizing chunks until the stopping criterion is met. Algorithm 1 summarizes the proposed heuristic.

Algorithm 1: The heuristic algorithm for solving ALP on single and multiple runways.

```

1 Input: Problem P; parameter  $k \in \mathbb{Z}^+$ ,  $k \ll n$ ; an initial sequence  $\pi$  of landings and its schedule  $S$ .
2 Output: An improved sequence and schedule of aircraft landings.
3 while the stopping condition is not met do
4    $\text{relax}(k)$  (relaxing the position of  $k$  aircraft in  $\pi$ );
5    $\pi', S' \leftarrow \text{optimize}(\pi)$  (optimizing the relaxed sequence by using an exact solver);
6    $\Delta \leftarrow \text{objective}(S') - \text{objective}(S)$ ;
7   if  $\Delta < 0$  then
8      $S \leftarrow S'$ ;
9      $\pi \leftarrow \pi'$ ;
10  end
11 end
12 return  $\pi, S$ 

```

An illustrative example

Let $\pi = (3, 4, 5, 6, 7, 8, 9, 1, 10, 2)$ represent a (initial) sequence of aircraft landings for an instance with 10 aircraft, which we presented earlier in Section 2.2. If we solve problem P with an exact solver by providing the solver with π , the solver can deliver an optimal landing schedule, i.e., the landing times for aircraft, because a known sequence implies that all variables y_{ij} and δ_{ij} are fixed to either 0 or 1. Therefore, problem P turns out to be a linear program. We note that this may not lead to the optimal solution for problem P. The optimal solution for problem P is obtained if we allow the solver to optimize both sequence and schedule of landings. Under this condition, it is clear that solving problem P to optimality is computationally expensive, particularly for large instances. The heuristic allows the solver to optimize only a chunk of aircraft, i.e., only a small number of aircraft is subject to optimization.

Assume $k = 6$, and the chunk therefore includes 6 aircraft. In the first iteration of the algorithm, we provide the solver with $\pi = (-, -, -, -, -, -, 9, 1, 10, 2)$, where the chunk of aircraft includes $(3, 4, 5, 6, 7, 8)$ and a “-” denotes a “relaxed” position. During optimizing problem P, aircraft, say 3, may no longer be the first aircraft in the landing sequence and it can be moved forward to occupy one of the available positions, highlighted by “-”. Similarly, aircraft 6, for example, may be moved backward or forward by the solver. In order to optimize the sequence and schedule of the chunk we may fix the binary variables y_{ij} to take a value of 1 for the aircraft that are not in the chunk. This leads to $y_{9,1} = 1$, $y_{1,10} = 1$ and $y_{10,2} = 1$. It is clear that the number of “actual” binary variables y_{ij} in problem P is now decreased, and hence, problem P may be solved easier. Also, note that the number of binary variables y_{ij} in problem P now depends on the size of the chunk.

In the second iteration, the heuristic chooses the chunk $(9, 1, 10, 2)$, which includes only 4 aircraft. The heuristic may change k for the last chunk if less than k aircraft are available. Solving problem P where only a chunk is optimized may lead to an improved sequence and schedule of aircraft landings. If an improved schedule is obtained, then π is updated accordingly.

4. Computational results

We tested the proposed heuristic on two sets of 124 benchmark instances. The first set includes 49 instances and the second one has 75 instances. We discuss these instances in Section 4.1. We use a set of 13 instances to tune the

parameters of the heuristic algorithm. We explain the parameter tuning in Section 4.2. We compare the proposed heuristic and two algorithms of Awasthi et al. [2] and Sabar and Kendall [14], and also the solver Gurobi under the time limit of 300 s. We selected the two algorithms of Awasthi et al. [2] and Sabar and Kendall [14] because they report high quality solutions, compared to the previous methods, for the first set of instances. On a single runway, Awasthi et al. [2] proposed an algorithm, which calculates the schedule for a given sequence of landings. On multiple runways, they developed a heuristic that assigns the aircraft to runways and determines the sequence of landings for each runway. Given such a sequence, their previous algorithm is applied to obtain the schedule. Sabar and Kendall [14] presented an iterated local search algorithm, in which a variable neighborhood descent algorithm is utilized as the local search. We did not compare our heuristic and the algorithm of Ghoniem and Farhadi [9] because the latter reported the outcomes within one hour of computation. In this regard, we note that the algorithm of Ghoniem and Farhadi [9] finds better solutions for the first set of instances than Awasthi et al. [2], Sabar and Kendall [14] and the heuristic of this study. No conclusion, however, can be made regarding the quality of the solution of the algorithm of Ghoniem and Farhadi [9] for the second set of instances as neither Awasthi et al. [2], nor Sabar and Kendall [14] did solve those instances. It should be noted that Girish [11] proposed several hybrid meta-heuristics for ALP and tested them on the first set that includes 49 instances. The study reports the best known solutions to date. Therefore, our proposed heuristic is not able to outperform the results by Girish [11]. Throughout the study we calculate the gap per instance as

$$\frac{z - z^*}{z^*} \times 100, z^* > 0$$

where z is the objective function value of a solution method (e.g., [2,14] or the proposed heuristic) and z^* is the best value for the objective function, which is obtained across all reported solution methods in this study. If both z and z^* are equal to 0, we let the gap be 0. We stop the heuristic algorithm either if the maximum computation time of 300 s is passed or the maximum number of iterations without improvement is recorded, whichever occurs the first.

We coded the heuristic algorithm in the programming language Python 2.7 and implemented the solver Gurobi version 7.5.1 [12] as the optimization solver. We ran the algorithm on a standard PC with an Intel®Core i5-6500 CPU at 3.20 GHz and 8GB of memory under Linux Ubuntu 16.04 LTS operating system. We use only one thread. Next, we discuss the instances, the details of the parameters tuning and the outcomes of the proposed heuristic algorithm for solving those 124 instances.

4.1. Instances

We solve two sets of instances. The first set is the well-known 49 standard benchmark instances of ALP, which are available from OR Library at <http://people.brunel.ac.uk/~mastjjb/jeb/info.html>. The instances range from 10 to 500 aircraft and 1 to 5 runways. In particular, 13 instances utilize one runway and the remaining 36 instances use between two and five runways. The characteristics of the instances include the earliest, latest and target landing times, and also earliness and lateness penalties per unit of deviation from the target landing time. We shall refer to these instances as set 1.

We also test the heuristic algorithm on 75 benchmark instances proposed in [9], which are available at <http://ahmed.ghoniem.info/download/MASP-SET.txt>. The instances range from 15 to 50 aircraft and 2 to 5 runways. We note that Ghoniem and Farhadi [9] presented a different formulation than problem P. The characteristic of instances is also slightly different than those of set 1. For example, each instance includes “ready time” and “due time” and represents both arrival and departure operations. We shall refer to these instances as set 2. To be able to solve the instances of set 2 by problem P and also by the heuristic algorithm, we replace the earliest landing time by the ready time and the target landing time by the due time. We set a very large value for the latest landing time.

4.2. Parameter tuning

There are two parameters of the heuristic algorithm that impact the algorithm’s performance. Those include the chunk size, i.e., the number of aircraft subject to optimization and the maximum number of optimization iterations allowed to perform without any improvement. We test four values of 8, 10, 12 and 15 for the chunk size and

Table 4

Value of parameters of the heuristic algorithm and the solver Gurobi.

Parameter	Notation	Value
Heuristic		
Chunk size	k	{8, 10, 12, 15}
Maximum iterations	$iter_{max}$	{5, 10, 15, 20}
Gurobi		
Time limit	τ	{10, 15, 20, 25} (seconds)
MIP absolute gap	–	0.01
MIP relative gap	–	0.01
MIP focus	f	{0, 2} (default, optimality)
Number of processor	–	1

Table 5The top two settings, out of 128 tested. The value of parameters is in the format $(\tau, k, iter_{max}, f)$.

z^*	Setting 1 (20, 10, 10, 2)			Setting 2 (15, 8, 15, 0)		
	z	Time (s.)	Gap (%)	z	Time (s.)	Gap (%)
700.00	700.00	0.29	0.00	700.00	0.24	0.00
1480.00	1480.00	0.94	0.00	1480.00	0.56	0.00
820.00	820.00	0.58	0.00	820.00	0.71	0.00
2520.00	2520.00	1.66	0.00	2520.00	1.09	0.00
3100.00	3100.00	2.80	0.00	3100.00	1.44	0.00
24442.00	24442.00	0.47	0.00	24442.00	0.66	0.00
1550.00	1550.00	1.16	0.00	1550.00	1.40	0.00
1950.00	1950.00	8.28	0.01	1950.00	8.77	0.00
5611.70	5611.99	32.04	0.00	5611.70	30.44	0.00
12292.20	12292.20	195.78	0.67	12336.61	219.82	0.36
12418.32	12501.49	116.36	0.08	12426.42	112.03	0.07
16209.78	16223.25	262.40	0.00	16300.01	251.61	0.56
39406.12	39406.12	315.06	0.00	39954.16	306.51	1.39
Average		72.14	0.06		71.94	0.18
Maximum		315.06	0.67		306.51	1.39

four values of 5, 10, 15 and 20 for the maximum number of iterations without improvement. For the Gurobi's parameters, we changed the default values of time limit, MIP focus, MIP absolute and relative gaps and the number of processors. We did not change the other parameters of Gurobi. For the first two parameters of Gurobi, i.e., time limit and MIP focus, we tested four values of 10, 15, 20 and 25 s for the time limit, and two values of 0 and 2 (default and focusing on optimality) for MIP focus. The value of 1 for MIP focus directs Gurobi to focus on feasibility, and hence, we did not test it. Table 4 shows the values for all parameters we tested. The motivation for testing these values of parameters is a trade-off between the solution quality and the solution time, which is in line with the aim of the study. For example, due to the penalty for both earliness and tardiness, moving an aircraft too far from its target landing time may not be beneficial.

It is easy to see that all combinations of those parameters lead to a set of 128 settings. We solve 13 instances of set 1, ranging from 10 to 500 aircraft and one runway and that per setting in order to evaluate those 128 settings. While it is impossible to include the summary of those 128 tests here due to the limited space, we report two settings in Table 5 that result in better solutions than other settings. We used two measures of “average gap” and “maximum gap” to evaluate the settings. We calculate the average and maximum gap of a setting over the 13 solved instances. We also report the average and maximum computation time of a setting over the 13 solved instances. Recall that we set the maximum computation time of the heuristic algorithm to 300 s. The two settings that we report in Table 5 show the smallest values of average and maximum gap over all 128 settings. Also, both have very close running times. We note that the average computation time of the 128 settings is close to one another, so we did not use this as an evaluation metric. We use these two settings in the heuristic algorithm to solve all benchmark instances.

Table 6

Summary of the solution methods for solving benchmark instances of set 1 with one runway.

Method	# of best solution	% of best solution
Setting 1	10	76.92
Setting 2	9	69.23
Awasthi et al. [2]	8	61.54
Sabar and Kendall [14]	11	84.62
Gurobi	8	61.54

Table 7

Detail of the solution methods for solving benchmark instances of set 1 with one runway.

n	z^*	Setting 1 z	Setting 2 z	Awasthi et al. [2] z	Sabar and Kendall [14] z	Gurobi z
10	700.00	700.00	700.00	700.00	700.00	700.00
15	1480.00	1480.00	1480.00	1480.00	1480.00	1480.00
20	820.00	820.00	820.00	820.00	820.00	820.00
20	2520.00	2520.00	2520.00	2520.00	2520.00	2520.00
20	3100.00	3100.00	3100.00	3100.00	3100.00	3100.00
30	24442.00	24442.00	24442.00	24442.00	24442.00	24442.00
44	1550.00	1550.00	1550.00	1550.00	1550.00	1550.00
50	1950.00	1950.00	1950.00	1950.00	1950.00	1950.00
100	5611.70	5611.99	5611.70	5703.54	5611.70	5642.19
150	12292.20	12292.20	12336.61	13515.68	12321.91	14151.15
200	12418.32	12501.49	12426.42	13401.57	12418.32	12793.83
250	16209.78	16223.25	16300.01	17346.45	16209.78	17397.35
500	39406.12	39406.12	39954.16	43052.04	41380.29	42265.70

4.3. Results for instances with one runway

Table 6 summarizes the outcomes of four methods of Awasthi et al. [2] and Sabar and Kendall [14], Gurobi (with a time limit of 300 s) and the heuristic of this study for 13 single runway instances of set 1. We evaluated the methods by using two criteria of “number of best solutions”, obtained by each method, and “percentage of best solutions”. Recall that we record the best solution among the four tested methods. As the table shows the best method is the algorithm of Sabar and Kendall [14], followed by the heuristic of this study. Table 7 details the outcomes of the four methods. From the outcomes, it is clear that the solver Gurobi obtains inferior solutions when the size of the instances becomes larger. We note that no single method among the four is shown to be able to deliver the best known solution for all instances. In addition, all methods are able to deliver the optimal solution for the instances with up to 50 aircraft. We may conclude that even though the heuristic algorithm of this study is not the top performing among those four for solving the instances of set 1 with one runway, it delivers quality solutions, particularly, the best known solutions for 10 instances and that within five minutes.

4.4. Results for instances with multiple runways

Table 8 summarizes the outcomes of four solution methods for solving 36 instances of set 1 and 75 instances of set 2 with multiple runways. From the table, it is clear that the proposed heuristic obtains a larger number of best solutions, compared to other methods. While the studies by Awasthi et al. [2] and Sabar and Kendall [14] did not solve instances of set 2, we can see from the results that the proposed heuristic delivers almost two times more best solutions than Gurobi, implying its effectiveness in solving those challenging instances of set 2. We note that due to the practical applications, as discussed in Section 1, it is very important that the heuristic of this study can deliver quality solutions in a short amount of time.

Tables 9 and 10 report detailed outcomes of the heuristic and Gurobi on instances of sets 1 and 2. Again, no single method is able to deliver all best known solutions.

Table 8

Summary of the solution methods for solving benchmark instances of sets 1 and 2 with multiple runways.

Method	Set 1		Set 2	
	# of best solution	% of best solution	# of best solution	% of best solution
Setting 1	34	94.44	33	44.00
Setting 2	31	86.11	50	66.67
Awasthi et al. [2]	29	80.56	Not solved	–
Sabar and Kendall [14]	25	69.44	Not solved	–
Gurobi	32	88.89	23	30.67

Table 9

Details of the solution methods for solving benchmark instances of set 1 with multiple runways.

<i>n</i>	<i>m</i>	<i>z</i> *	Setting 1 <i>z</i>	Setting 2 <i>z</i>	Awasthi et al. [2] <i>z</i>	Sabar and Kendall [14] <i>z</i>	Gurobi <i>z</i>
10	2	90.00	90.00	90.00	90.00	90.00	90.00
	3	0.00	0.00	0.00	0.00	0.00	0.00
15	2	210.00	210.00	210.00	210.00	210.00	210.00
	3	0.00	0.00	0.00	0.00	0.00	0.00
20	2	60.00	60.00	60.00	60.00	60.00	60.00
	3	0.00	0.00	0.00	0.00	0.00	0.00
20	2	640.00	640.00	640.00	640.00	640.00	640.00
	3	130.00	130.00	130.00	130.00	130.00	130.00
20	4	0.00	0.00	0.00	0.00	0.00	0.00
	2	650.00	650.00	650.00	650.00	650.00	650.00
30	3	170.00	170.00	170.00	170.00	170.00	170.00
	4	0.00	0.00	0.00	0.00	0.00	0.00
30	2	554.00	554.00	554.00	554.00	554.00	554.00
	3	0.00	0.00	0.00	0.00	0.00	0.00
44	2	0.00	0.00	0.00	0.00	0.00	0.00
	2	135.00	135.00	135.00	135.00	135.00	135.00
50	3	0.00	0.00	0.00	0.00	0.00	0.00
	2	444.10	444.10	444.10	444.10	445.04	444.10
100	3	75.75	75.75	75.75	75.75	75.75	75.75
	4	0.00	0.00	0.00	0.00	0.00	0.00
150	2	1143.7	1143.7	1143.7	1203.76	1271.07	1143.70
	3	205.21	205.21	205.21	205.21	205.21	205.21
150	4	34.22	34.22	37.44	34.22	34.22	34.22
	5	0.00	0.00	0.00	0.00	0.00	0.00
200	2	1330.91	1330.91	1345.49	1400.64	1410.02	1349.96
	3	253.07	253.07	253.07	253.15	271.10	253.07
200	4	54.53	54.53	54.53	54.53	54.53	54.53
	5	0.00	0.00	0.00	0.00	0.00	0.00
250	2	1702.33	1750.09	1702.33	1753.67	1961.39	1719.46
	3	221.97	221.97	221.97	233.49	272.03	221.97
250	4	2.44	2.44	2.44	2.44	3.40	2.44
	5	0.00	0.00	0.00	0.00	0.00	0.00
500	2	4354.53	4354.53	4490	4593.77	5458.49	4422.79
	3	682.69	682.69	723.73	712.81	1108.51	713.40
500	4	89.95	92.37	92.37	89.95	92.89	89.95
	5	0.00	0.00	0.00	0.00	3.00	0.00

Overall, the outcomes show that the heuristic of this study is able to deliver quality solutions even for large instances and that within a short amount of time, implying it can be used to optimize scheduling aircraft arrivals and departures at busy airports.

5. Conclusion

This study investigated a new approach to solve the aircraft landing problem (ALP). We developed a heuristic algorithm, which is shown to be capable of delivering high quality solutions in a short time. We compared the

Table 10

Details of the solution methods for solving benchmark instances of set 2 with multiple runways.

n	m	z^*	Setting 1 z	Setting 2 z	Gurobi z
15	2	6396	6396	6396	6396
15	2	9780	9780	9780	9780
15	2	3000	3000	3000	3000
15	2	7560	7560	9240	7560
15	2	27840	27840	29340	27840
15	3	25200	25200	25440	25200
15	3	14880	16860	14880	14880
15	3	18300	18300	18300	18600
15	3	19500	19500	19500	19500
15	3	28299	29085	28299	28299
15	4	41280	41280	41280	41280
15	4	45180	45900	45360	45180
15	4	45496	47748	45496	46186
15	4	37836	38022	38022	37836
15	4	39960	40050	39960	41220
20	2	0	0	0	0
20	2	63180	66300	63180	82200
20	2	15390	15390	15390	15390
20	2	4620	4620	4620	7320
20	2	0	0	0	0
20	3	54240	54240	54900	57960
20	3	90180	90180	90360	96780
20	3	75660	75660	75840	78240
20	3	118462	118462	129613	125959
20	3	106980	112650	106980	114420
20	4	115080	115080	118320	116640
20	4	127500	128280	127500	130200
20	4	152103	162460	152103	166975
20	4	134791	134791	134791	138031
20	4	154810	154810	155299	187217
20	5	163224	170460	163224	169625
20	5	126540	128460	126540	130800
20	5	189080	189080	189264	190935
20	5	172947	176229	172947	175923
20	5	208475	208475	209118	209121
25	2	2040	2280	2040	2040
25	2	4080	4080	4080	4080
25	2	6210	9018	6210	6210
25	2	39720	43380	40980	39720
25	2	0	0	0	0
25	3	221940	224640	221940	248340
25	3	160631	160631	160631	171974
25	3	121800	139380	121800	129360
25	3	123126	134799	123126	150621
25	3	128880	141072	130272	128880
25	4	224820	239820	224820	231900
25	4	291342	296164	291342	302226
25	4	223200	225780	223200	231780
25	4	303872	338181	303872	323509
25	4	181200	181200	183720	186960
25	5	292217	292217	307303	306858
25	5	263160	265080	276900	263160
25	5	331923	331923	332406	332220

(continued on next page)

Table 10 (continued).

n	m	z^*	Setting 1 z	Setting 2 z	Gurobi z
25	5	280620	284220	284700	280620
25	5	290760	300360	290760	291900
50	2	82748	82748	87923	157014
50	2	207420	207420	233340	281880
50	2	199380	200100	199380	246360
50	2	176280	191400	176280	210060
50	2	44055	44055	44385	67812
50	3	944760	944760	945060	1011660
50	3	837780	862980	837780	894240
50	3	1260896	1332394	1260896	1327052
50	3	873890	899401	873890	943610
50	3	890460	904860	890460	934740
50	4	1817010	1826370	1817010	2073120
50	4	1928251	1972484	1928251	1999395
50	4	1647000	1689420	1647000	1750500
50	4	1551780	1605000	1551780	1627020
50	4	2961288	2961288	2988021	3151229
50	5	1943940	1975860	1943940	2101620
50	5	2022960	2059380	2022960	2115780
50	5	1260896	1332394	1260896	1327052
50	5	1847940	1855920	1847940	1946220
50	5	2280720	2321310	2280720	2529450

proposed heuristic algorithm and two heuristic algorithms from the literature, and also an exact solver. We tested the algorithm on 124 instances and showed that while the heuristic is quick enough to be implemented in practice, it also delivers quality solutions. We believe the most important advantages of our algorithm include its simplicity, solution quality and running time. These characteristics contribute to usability of the algorithm for practical applications.

Acknowledgments

We thank the editors and anonymous referees for their helpful comments on the earlier version of this paper. Amir Salehipour is the recipient of an Australian Research Council Discovery Early Career Researcher Award (project number DE170100234) funded by the Australian Government.

References

- [1] J. Abela, D. Abramson, M. Krishnamoorthy, A. De Silva, G. Mills, Computing optimal schedules for landing aircraft, in: Proceedings of the 12th national conference of the Australian Society for Operations Research, Adelaide, 1993, pp. 71–90.
- [2] A. Awasthi, O. Kramer, J. Lässig, Aircraft landing problem: An efficient algorithm for a given landing sequence, in: 2013 IEEE 16th International Conference on Computational Science and Engineering, 2013, pp. 20–27.
- [3] H. Balakrishnan, B.G. Chandran, Chandran algorithms for scheduling runway operations under constrained position shifting, *Oper. Res.* 58 (6) (2010) 1650–1665.
- [4] J.E. Beasley, M. Krishnamoorthy, Y.M. Sharaiha, D. Abramson, Scheduling aircraft landings—the static case, *Transp. Sci.* 34 (2) (2000) 180–197.
- [5] J.A. Bennell, M. Mesgarpour, C.N. Potts, Airport runway scheduling, *4OR* 9 (2) (2011) 115.
- [6] L. Bianco, P. Dell’Olmo, S. Giordani, Minimizing total completion time subject to release dates and sequence-dependent processing times, *Ann. Oper. Res.* 86 (1999) 393–415.
- [7] A.T. Ernst, M. Krishnamoorthy, R.H. Storer, Heuristic and exact algorithms for scheduling aircraft landings, *Networks* 34 (3) (1999) 229–241.
- [8] F. Furini, M.P. Kidd, C.A. Persiani, P. Toth, Improved rolling horizon approaches to the aircraft sequencing problem, *J. Sched.* 18 (5) (2015) 435–447.
- [9] A. Ghoniem, F. Farhadi, A column generation approach for aircraft sequencing problems: a computational study, *J. Oper. Res. Soc.* 66 (10) (2015) 1717–1729.
- [10] A. Ghoniem, F. Farhadi, M. Reihaneh, An accelerated branch-and-price algorithm for multiple-runway aircraft sequencing problems, *European J. Oper. Res.* 246 (1) (2015) 34–43.

- [11] B. Girish, An efficient hybrid particle swarm optimization algorithm in a rolling horizon framework for the aircraft landing problem, *Appl. Soft Comput.* 44 (2016) 200–221.
- [12] I. Gurobi Optimization Inc., Gurobi Optimizer Reference Manual, 2016.
- [13] H. Pinol, J. Beasley, Scatter search and bionomic algorithms for the aircraft landing problem, *European J. Oper. Res.* 171 (2) (2006) 439–462.
- [14] N.R. Sabar, G. Kendall, An iterated local search with multiple perturbation operators and time varying perturbation strength for the aircraft landing problem, *Omega* 56 (2015) 88–98.
- [15] A. Salehipour, M.M. Ahmadian, A heuristic algorithm for the Aircraft Landing Problem, in: *The 22nd International Congress on Modelling and Simulation (MODSIM2017)*, 2017.
- [16] A. Salehipour, M.M. Ahmadian, D. Oron, Efficient and simple heuristics for the Aircraft Landing Problem, in: *Matheuristic 2018 International Conference*, 2018.
- [17] A. Salehipour, M. Modarres, L.M. Naeni, An efficient hybrid meta-heuristic for aircraft landing problem, *Comput. Oper. Res.* 40 (1) (2013) 207–213.
- [18] A. Salehipour, L.M. Naeni, H. Kazemipoor, Scheduling aircraft landings by applying a variable neighborhood descent algorithm: Runway-dependent landing time case, *J. Appl. Oper. Res.* 1 (1) (2009) 39–49.
- [19] S. Vadlamani, S. Hosseini, A novel heuristic approach for solving aircraft landing problem with single runway, *J. Air Transp. Manag.* 40 (2014) 144–148.
- [20] M. Wen, J. Larsen, J. Clausen, An exact algorithm for aircraft landing problem, Department of Informatics and Mathematical Modeling, Technical University of Denmark, 2005.